# Becker Article Automation Pipeline

### Migration to GitHub Self-Hosted Runner

---

## About This Document

**Purpose:** This is the reference document for the Becker Article Automation Pipeline migration. It covers the full story: from the original manual process, through the automation architecture, to the migration of the runner from GitHub cloud to an on-premises server. It also logs every challenge encountered during implementation and how each was resolved.

**What is in this document:**

- Background – Why the pipeline was built and what problem it solves
- Pipeline architecture – How Stage 1, 2 and 3 work together
- Migration rationale – Why GitHub self-hosted runner was chosen
- Implementation blueprint – Phase-by-phase setup instructions
- Challenges log – Every issue encountered, step by step, with resolutions
- Server selection rationale – Which server was chosen and why
- People of contact – Who owns what
- Final YAML configuration – The production-ready workflow file

---

## 1. Background – From Manual to Automated

### 1.1 The Original Manual Process

Before automation, the Becker article extraction and summarisation process was entirely manual:

- Articles were tracked and downloaded by hand from Becker's website
- Stage filtering (removing irrelevant articles) was done manually by the Product Analyst
- LLM-based classification and summarisation was run ad-hoc on a local machine
- Output files were saved locally, with no consistent version control or scheduling

This approach was time-consuming, error-prone and dependent on individual availability.

### 1.2 The Automation Goal

The goal was to build a fully automated, scheduled pipeline that:

- Runs without any human trigger on every Friday at 9:00 PM UTC
- Scrapes all Becker article sections including Finance, which is behind anti-bot protection
- Filters articles using keyword rules (Stage 2) before passing to LLM
- Uses an LLM to classify and summarise articles (Stage 3)
- Saves outputs as CSV files and commits them back to the GitHub repository automatically

### 1.3 The Core Problem – Finance Section on Cloud Runners

When the pipeline was first hosted on GitHub Actions using cloud runners (ubuntu-latest), Stages 2 and 3 ran correctly, but Stage 1 failed to scrape Finance articles.

The reasons:

- **Datacenter IP blocking:** Cloud providers (Azure, AWS, GCP) publish their IP ranges. Finance sites maintain blocklists and reject requests from these ranges before any content is served.
- **Browser fingerprinting:** Headless Chromium on a freshly provisioned cloud VM produces a distinct fingerprint that anti-bot systems detect and flag.
- **JavaScript challenges:** Finance platforms (Cloudflare Turnstile, PerimeterX) analyse rendering behaviour and Cloud VMs fail these checks.

The same code ran without issue on the local machine because it carried a trusted corporate IP not in any cloud-provider CIDR block and its pre-installed Chrome produces a more authentic browser signature.

---

## 2. Pipeline Architecture

### 2.1 How the Pipeline Works

- **Stage 1 – Web Scraping:** Selenium-based scraper pulls articles from Becker's website including Finance sections. Outputs a delta CSV (only new articles since last run).

- **Stage 2 – Keyword Filter:** Applies keep/exclude regex patterns to reduce the dataset before LLM processing. Saves both kept and removed rows for QC.
- **Stage 3 – LLM Classification:** Sends filtered articles to an LLM via OpenRouter API for classification and summarisation. Outputs structured CSV files.
- **Commit & Push:** All output folders (OUTPUT_STAGE1, OUTPUT_STAGE2, OUTPUT_STAGE3) are committed and pushed back to GitHub after each run.
- **Trigger:** GitHub Actions cron schedule fires every Friday at 21:00 UTC. Can also be triggered manually via workflow_dispatch.

### 2.2 Code Files

- stage1_code.py – Selenium scraper (web scraping with Chrome)
- stage2_code.py – Keyword delta filter
- stage3_code.py – LLM classification and output
- scraper.yml – GitHub Actions workflow definition code
- requirements.txt – Python dependencies (pandas, selenium, webdriver-manager, requests, beautifulsoup4, python-dateutil)
- .gitignore – Excludes __pycache__, .pyc, venv, .env from version control

---

## 3. Migration Solution – GitHub Self-Hosted Runner

### 3.1 What Is a Self-Hosted Runner?

A self-hosted runner is a lightweight GitHub agent installed on the server that:

- Polls GitHub over outbound HTTPS, waiting for jobs
- When a cron fires or workflow is triggered manually, GitHub dispatches the job to this agent instead of provisioning a cloud VM
- Executes all pipeline steps locally on the server
- Reports logs, exit codes and artefact paths back to GitHub Actions exactly as cloud runners do

**Key principle:** GitHub handles scheduling and coordination. The server handles execution. Nothing in the pipeline, scheduling, secrets, or output logic changes.

### 3.2 Why Self-Hosted Runner Over Other Options

- Free – Self-hosted runners are completely free and have no monthly minute cap
- Minimal change – Only one line changes in the YAML (runs-on: self-hosted)
- Trusted IP – The server's corporate IP is not in any cloud-provider blocklist
- Pre-installed dependencies – Chrome, ChromeDriver and Python are installed once, not per run
- Same scheduling – The Friday 21:00 UTC cron trigger continues to work exactly as before

---

## 4. Server Selection & People of Contact

### 4.1 Server Selection Rationale

Two environments were evaluated before settling on the production server:

| Server | OS | Admin Privileges | Reason Selected |
|---|---|---|---|
| Local Machine (Test Phase) | Windows | No | Used only for initial validation. Not suitable for permanent service due to no admin access and no 24/7 uptime. |
| Server 159 (Production) | Windows Server | Yes. (Pre-configured by IT) | Recommended by Keshav. Comes with admin privileges and 24/7 uptime. Eliminates permission and registry issues encountered on local machine. |

**Decision:** Server 159 was recommended by Keshav. It was chosen because it already has admin privileges configured, is on 24/7 and has no existing runner conflicts. This avoids all the permission and registry issues encountered during local machine testing.

### 4.2 People of Contact

| Name | Role | Area of Responsibility | When to Contact |
|---|---|---|---|
| Sherwin Mathias | Senior Product Analyst | Scoping, approval and documentation requirements | Change requests, scope additions |
| Keshav Kant | IT System Administrator | Server access, admin privileges, runner service setup, firewall configuration | Server-side issues, permissions, registry errors, Chrome/Python/ Git updates |

# 5. Implementation Blueprint

The migration is structured across four phases. Phases 1 and 2 are one-time server-side tasks. Phase 3 is a single-line code change. Phase 4 requires no action.

## Phase 1 – Register the Self-Hosted Runner

### Step 1 : Generate the Registration Token (GitHub UI)

Navigate in GitHub to: Repository Settings → Actions → Runners → New self-hosted runner

- Select the server OS (Windows for Server 159)
- GitHub generates a short-lived registration token and the exact binary download URL

**NOTE:** The token expires within one hour – run the next step immediately after generating it.

### Step 2 : Download and Configure the Runner (PowerShell – Run as Administrator)

Create a folder for the runner and navigate into it:

```
mkdir actions-runner
cd actions-runner
```

Download the runner package using the URL provided by GitHub, then extract it. Then run:

```
./config.cmd --url https://github.com/YOUR_ORG/YOUR_REPO --token YOUR_TOKEN
```

During configuration you will be prompted for five inputs. Press Enter to accept defaults for all of them:

- Runner group – Press Enter
- Runner name – Press Enter
- Runner labels – Press Enter

- Work folder – Press Enter
- Run as service? – Type Y and press Enter (for production server)
- Service account – Press Enter to accept NT AUTHORITY\NETWORK SERVICE

### Step 3 : Confirm the Service Is Running

After answering Y to the service question, the runner installs and starts automatically. Confirm it is running:

Open Windows Services (Windows key → type Services) and look for:

```
actions.runner.[ORG]-[REPO].[MACHINE_NAME]
```

Status should show: Running

Alternatively, check via PowerShell:

```
Get-Service | Where-Object {$_.Name -like "*actions*"}
```

**NOTE:** Once confirmed as Running, PowerShell can be closed. The service runs in the background 24/7 and survives reboots automatically.

## Phase 2: One-Time Dependency Setup on the Server

Dependencies must be pre-installed on the server. This is done once and persists across all future runs.

### Python Packages

```
pip install -r requirements.txt
```

### Chrome (if not already installed)

Download and install Google Chrome from:

```
https://www.google.com/chrome/
```

Verify installation:

```
google-chrome --version   (or check via Start Menu)
```

### ChromeDriver

webdriver-manager handles ChromeDriver automatically. Ensure it is in requirements.txt (it is). No manual ChromeDriver download is needed.

Chrome version maintenance: If a Windows Update or IT patch updates Chrome, webdriver-manager will auto-download the matching ChromeDriver on the next run. No manual action needed. Contact Keshav if Chrome updates are managed centrally.

## Phase 3 : YAML Configuration Change

This is the only change required to the repository code:

### Change

**Old (cloud runner):**

```
runs-on: ubuntu-latest
```

**New (self-hosted):**

```
runs-on: self-hosted
```

### Steps to Remove

Since dependencies are pre-installed on the server, remove these commented steps from the YAML:

- Install Google Chrome – Remove entirely
- Install matching ChromeDriver – Remove entirely
- Set up Python – Remove entirely

### Steps to Retain

- Checkout Repository – Keep as is
- Check Python – Keep (validates Python is available)
- Install Dependencies – Keep (pip install picks up any future requirements.txt changes)
- Run Stage 1, Stage 2, Stage 3 – Keep as is
- Commit and Push Outputs – Keep with Windows-compatible syntax (see below)

## Phase 4 : Secrets & Credentials (No Action Required)

- OPENROUTER_API_KEY – No change
- MODEL – No change
- OPENROUTER_URL – No change

The manual code stored these credentials in an .env file. However, since .env files cannot be stored on the repository, since they are publicly exposed, thus also exposing the secret credentials in the process. Hence, they are stored in the Secrets part of Repository.

Navigate in GitHub to: Repository Settings → Secrets and variables → Actions → Repository secrets → New repository secret

Secrets are never stored on the server. The runner fetches them from GitHub at job start and exposes them only for the duration of that job.

---

## 6. Final YAML Configuration (Production-Ready)

The following is the complete, validated scraper.yml for Windows self-hosted runner. This is the file currently in the repository:

```
name: Weekly Automation Pipeline


on:
  schedule:
    - cron: "0 21 * * 5" # Runs every week on Friday at 9 pm
  workflow_dispatch:


permissions:
  contents: write


jobs:
  run-pipeline:
    runs-on: self-hosted


    env:
      OPENROUTER_API_KEY: ${{ secrets.OPENROUTER_API_KEY }}
      MODEL: ${{ secrets.MODEL }}
      OPENROUTER_URL: ${{ secrets.OPENROUTER_URL }}


    steps:
```

```
      - name: Checkout Repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Check Python
        run: python --version

      - name: Install Dependencies
        run: pip install -r requirements.txt

      - name: Run Stage 1
        run: python stage1_code.py

      - name: Run Stage 2
        run: python stage2_code.py

      - name: Run Stage 3
        run: python stage3_code.py

      - name: Commit and Push Outputs
        run: |
          git config --global user.name "github-actions"
          git config --global user.email "github-actions@github.com"
          git add OUTPUT_STAGE1/ OUTPUT_STAGE2/ OUTPUT_STAGE3/
          $changes = git status --porcelain
          if ($changes) {git commit -m "Update pipeline outputs"}
          git push
```

# 7. Implementation Challenges Log

This section documents every challenge encountered during the implementation, in the order they appeared. Each entry includes the step at which it occurred, the exact error or symptom and how it was resolved.

Testing was first performed on a local Windows machine (restricted office computer) before migrating to Server 159.

### Challenge 1 : PowerShell Script Execution Blocked (ExecutionPolicy)

| | |
|---|---|
| **Challenge** | Pipeline failed with: 'setup.ps1 cannot be loaded because running scripts is disabled on this system.' The runner's internal setup scripts were blocked by Windows default script execution policy. |
| **Resolution** | Opened PowerShell as Administrator and ran: Set-ExecutionPolicy RemoteSigned -Scope LocalMachine, then confirmed with Y. Restarted the runner and re-triggered the workflow. This is a one-time setup per machine. |

### Challenge 2 : Registry Access Denied (No Admin Rights on Local Machine)

| | |
|---|---|
| **Challenge** | After resolving the execution policy, pipeline failed with: 'Remove-Item: Requested registry access is not allowed.' The runner's setup.ps1 script attempted to write to Windows registry entries that required administrator privileges. The local machine was a restricted office computer with no admin access. |
| **Resolution** | This was a hard wall as registry writes cannot be bypassed without admin rights. Decision made to proceed with local machine testing only in manual mode (N to service installation), validate the pipeline end-to-end and then migrate to Server 159 which has full admin privileges. Keshav assisted in identifying Server 159 as the correct production target. |
| **Contact** | Keshav |

### Challenge 3 : Git Author Identity Not Set

| | |
|---|---|
| **Challenge** | When running git commit for the first time from the runner, it failed with: 'Author identity unknown. Please tell me who you are.' Git had no global user name or email configured on this machine. |

| | |
|---|---|
| **Resolution** | Ran the following two commands once in PowerShell: git config --global user.email "github-actions@github.com" git config --global user.name "github-actions" These are also included in the YAML commit step as double security to ensure it always runs in any environment. |

### Challenge 4 : Git Push Rejected (Remote Had Newer Changes)

| | |
|---|---|
| **Challenge** | git push failed with: 'Updates were rejected because the remote contains work that you do not have locally.' The repository had received new commits after the local clone was made. |
| **Resolution** | Ran: git pull --rebase This fetched the latest remote changes and rebased the local commit on top. Then ran git push, which succeeded cleanly. |

### Challenge 5 : Inter-Dependency Clashes Between Runner and Existing Server Packages

| | |
|---|---|
| **Challenge** | During pip install -r requirements.txt on Server 159, some packages conflicted with versions already present in the server's Python environment. This caused import errors during Stage 1 execution. |
| **Resolution** | Worked with Keshav to identify the conflicting packages. Ran pip install with --upgrade flag for the specific packages causing conflicts. Confirmed clean execution by running a manual test workflow before finalising the service configuration. |
| **Contact** | Keshav |

## 8. Before vs After Comparison

| Aspect | Before (Cloud Runner) | After (Self-Hosted Runner) |
|---|---|---|
| **Execution host** | GitHub VM – New each run | On-premises server – Persistent |

| Aspect | Before (Cloud Runner) | After (Self-Hosted Runner) |
|---|---|---|
| **IP address** | Datacenter range – Blocked by Finance sites | Corporate server IP – Trusted, not blocked |
| **Finance scraping** | FAILS – IP blocked at request level | SUCCEEDS – Trusted IP passes anti-bot checks |
| **Chrome install overhead** | ~2–3 min per run | 0 sec – Pre-installed on server |
| **Scheduling** | GitHub Actions cron | Same – Unchanged |
| **Secrets management** | GitHub Secrets | Same – Unchanged |
| **Output commit/push** | git push to GitHub repo | Same – Unchanged |
| **Cost** | Free (within monthly minutes cap) | Free – Self-hosted runners are unlimited |

# 9. Operational Notes

## 9.1 What Happens Every Week

- Friday 21:00 UTC – GitHub triggers the pipeline automatically via cron
- Within seconds – Server 159 runner picks up the job
- Stage 1 runs – Scrapes all Becker articles including Finance sections (~15 minutes total for all 3 stages)
- Stage 2 runs – Filters delta using keyword rules
- Stage 3 runs – LLM classification and output
- Commit and Push – Output CSVs committed to GitHub repository

## 9.2 Monitoring

- GitHub Actions tab shows live run status and full step-by-step logs
- GitHub sends email notification to the repository owner if a run fails
- Server 159's runner service can be checked at any time via Windows Services or PowerShell (Get-Service | Where-Object {$_.Name -like "*actions*"})

### 9.3 Maintenance Responsibilities

- Chrome version updates : Webdriver-manager handles ChromeDriver auto-download. No manual action needed unless Chrome's major version changes in a way that breaks the driver. If issues arise, contact Keshav.
- Python package updates : Run pip install -r requirements.txt. Upgrade on Server 159 if new packages are added to requirements.txt.
- GitHub runner token : The registration token is a one-time use during setup only. It does not need renewal once the runner is configured.
- Runner service restart : If the service shows Stopped in Windows Services, right-click → Start. If it stops repeatedly, check GitHub Actions logs for job errors.

### 9.4 Important Notes for Future Reference

- The runner registration token expires within one hour. If you ever need to re-register the runner, generate a fresh token from GitHub Settings → Actions → Runners → New self-hosted runner immediately before running config.cmd.
- The || operator does not work in Windows PowerShell. Any future YAML edits that use bash conditional syntax (||, &&, if [...]) must be rewritten in PowerShell syntax.
- If git push is ever rejected, run git pull --rebase first, then push again. This handles cases where the repository received new commits between the runner's checkout and its push.

---

## References

**[1]** GitHub Docs – About self-hosted runners https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners

**[2]** GitHub Docs – Adding a self-hosted runner https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/adding-self-hosted-runners

**[3]** GitHub Docs – Configuring the runner application as a service https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/configuring-the-self-hosted-runner-application-as-a-service

**[4]** GitHub Docs – Monitoring and troubleshooting self-hosted runners
https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/monitoring-and-troubleshooting-self-hosted-runners

**[5]** GitHub Docs – Security hardening for GitHub Actions
https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions

**[6]** Microsoft Docs – PowerShell ExecutionPolicy https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/set-executionpolicy

**[7]** Selenium Docs – WebDriver troubleshooting
https://www.selenium.dev/documentation/webdriver/troubleshooting/errors/driver_location/

**[8]** PyPI – webdriver-manager documentation https://pypi.org/project/webdriver-manager/

**[9]** Chrome for Testing – ChromeDriver version resolution
https://googlechromelabs.github.io/chrome-for-testing/

**[10]** Cloudflare Blog – Bot Management and datacenter IP detection
https://blog.cloudflare.com/cloudflare-bot-management-machine-learning-and-more/

**[11]** GitHub Community – Self-hosted runners behind corporate proxy
https://github.com/orgs/community/discussions/26288

**[12]** Microsoft Docs – Windows Services administration https://learn.microsoft.com/en-us/windows/win32/services/services